Semantic Search

Modulo 2 · Lezione 8



Da «misurare coppie» a «cercare»

Nella lezione scorsa il testo è diventato numeri e avete salvato il corpus **vettorizzato** su disco (`corpus_embeddings.npz` : 48 testi → 48 vettori). Ma misurare coppie *scelte a mano* non è cercare.

`Modulo 2 · Lezione 7`

Similarità di **2 testi scelti da noi**: `cosine(a, b)` .
Sapevamo già *cosa* confrontare.

Modulo 2 · Lezione 8 — stasera

UNA domanda contro TUTTO il corpus: i k testi più vicini, ordinati per similarità. E i risultati vanno *ordinati*, *filtrati*, *diversificati*.

Niente database, stasera: **numpy puro**. Si capisce COSA fa un retrieval prima di delegarlo (nella prossima lezione).



Agenda

Quando c'è da lavorare, una **slide di switch** vi manda al posto giusto — finita l'attività, si torna qui.
🎤 = si legge/scrive col docente · ⌨️ = scrivete voi (docente in affiancamento)

	Blocco	Dove si lavora	Tempo
📊	Teoria: anatomia della ricerca, MMR, threshold, debug, scala	slide	40′
🎤	**Lettura guidata: si riapre il corpus**	`working/search.py`	10′
⌨️	**Esercizio 1: scrivete search e search_mmr**	`working/search.py`	40′
⌨️	**Esercizio 2: il query round** (query dal docente)	il vostro `search.py`	25′
📊	Wrap-up: le fasce + il toolkit	slide	5′

La chiave è **quella di Modulo 2 · Lezione 1 – Modulo 2 · Lezione 7**, nessuna registrazione nuova (persa? **alzate la mano**: c'è la riserva del docente). Il corpus è quello della lezione scorsa: il **vostro** — o la copia di riserva già in `dataset/` . Costo dell'intera serata: **~$0.000002** di query.

Setup (venv, requirements, `.env`): passo-passo in `scripts/CONSEGNA.md` — si fa **nei primi minuti della teoria** (se `scripts/.venv` esiste già, è fatto). Tutti i comandi si lanciano **dalla cartella** `scripts/` .

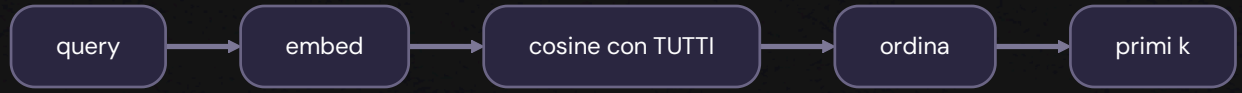


1. L’anatomia della ricerca



Il giro completo: Modulo 2 · Lezione 7, ripetuta 48 volte

La ricerca semantica non ha magia: è la **cosine di Modulo 2 · Lezione 7**, ripetuta per tutto il corpus, più un ordinamento.



La STESSA funzione della lezione scorsa, una volta per testo

```
scores = [cosine_similarity(q, v) for v in vectors]
```

Coi nostri 48 testi: **0.75 ms**. Poi si ordina — `risultati.sort(key= ... , reverse=True)` , Python che sapete già — e i primi k sono serviti.

> L'ordinamento tocca **tutto il corpus** per restituirvi k risultati. Segnatevelo: torna tra qualche slide, quando parliamo di scala.

Query e corpus nello stesso spazio

Con quale modello embeddiamo la **query**? Risponde `corpus_meta.json` — il **certificato di origine** salvato nella lezione scorsa insieme ai vettori:

```
{ "backend": "openai", "modello": "text-embedding-3-small", "dimensioni": 1536, ... }
```

Spazi diversi = nonsense

Corpus embeddato con `openai` (1536 dim), query con `local` (384 dim): non «risultati un po' peggiori» — **numeri senza senso**, o un crash. È la trappola classica di chi mischia i backend tra ingestion e query.

Coerenza per costruzione

Per questo `search()` **legge il backend dal corpus** (`carica_corpus` glielo restituisce) e embedda la query con quello: query e corpus sono sempre nello stesso spazio.

> Vale anche **nel tempo**: se il fornitore cambia modello, i vettori vecchi non si confrontano coi nuovi (Modulo 2 · Lezione 7, criterio stabilità).



Top-k e i suoi difetti

Top-k = i k score più alti, in ordine. Semplice — e con due difetti strutturali che stasera tocchiamo con mano.

Difetto 1 — si riempie COMUNQUE

k risultati escono **sempre**, anche per una query assurda: la lista piena *non significa* risposta trovata. Qualcosa deve dire «questi non valgono niente».

Difetto 2 — i quasi-doppioni si rubano i posti

Se 5 testi dicono la *stessa cosa*, il top-5 è un **monologo**: rilevantissimo, e inutile dal secondo risultato in poi.

Due difetti → due manopole

threshold (butta fuori la spazzatura) e **MMR** (compra diversità). Sono il capitolo 2 — e stasera le usate entrambe.

> C'è un terzo difetto — score alti ma risultati *sbagliati* — che non è del top-k ma dell'**embedding**: la trappola di Modulo 2 · Lezione 7. La rivedrete stasera, dal lato ricerca.



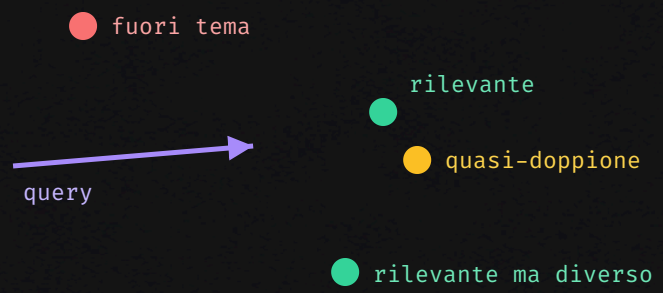
2. Le manopole: MMR, threshold, debug



MMR: rilevanza senza doppioni

Maximal Marginal Relevance: si sceglie un risultato alla volta — ogni giro vince chi è *rilevante* ma *diverso* dai già scelti.

$$\lambda \cdot \text{sim}(\text{cand}, \text{query}) - (1-\lambda) \cdot \max \text{sim}(\text{cand}, \text{già scelti})$$



top-2: rilevante + quasi-doppione — un monologo
MMR (λ=0.5): rilevante + diverso — **il quasi-doppione perde il posto**

λ è la manopola

A **λ = 1.0** resta solo la rilevanza: MMR **È** il top-k. A λ = 0 conta solo la diversità. In mezzo si compra diversità *pagando* un po' di rilevanza.

A mano: cicli for e la cosine di Modulo 2 · Lezione 7

Rilevanza = `cosine_similarity(v, query)` per ogni testo; il primo scelto è il più rilevante, poi un ciclo che ogni giro premia il miglior compromesso tra rilevanza e distanza dai già scelti. La scrivete tra poco — niente algebra.

Threshold: gli score si leggono a fasce

Quando uno score è «abbastanza buono»? La risposta sbagliata: un numero copiato da un tutorial.

Gli score NON sono calibrati

0.6 non è «buono» in assoluto: dipende da **modello** e **corpus**. Modulo 2 · Lezione 7 docet: 0.6 di `openai` e 0.6 di MiniLM non sono la stessa cosa.

Il metodo: misurare le fasce

Lanci query di **famiglie diverse** — sensate, vaghe, fuori dominio — e guardi dove cadono i massimi. La soglia va nel **vuoto** tra la fascia buona e la spazzatura.

Stasera la soglia la scegliete VOI

Nei round misurate le fasce sul **vostro** corpus e scegliete la **vostra** soglia: un numero che vi portate a casa — nel codice di stasera non c'è, servirà quando il retrieval finirà in un'applicazione. Con un altro modello andrà ritarata.

> Applicarla, quando servirà, è un confronto: score sotto la soglia → risultato **FUORI**. La vostra la scegliete a fine serata, dalle fasce dei round.



Debug del retrieval: i sintomi

Una ricerca «che va male» va **diagnosticata**, non subita. Gli score sono i sintomi:

Sintomo	Diagnosi	Cura
Score **alti e staccati** (0.7+, poi il vuoto)	query sana	niente: funziona
Score **piatti e mediocri** (tutti ~0.5)	query **vaga**	riformulare: più specifica
Score **tutti bassi** (sotto ~0.3)	**fuori dominio**	il threshold li butta fuori
Score alti ma risultati **sbagliati**	trappola semantica (negazioni: **tema ≠ verità**)	saperlo — il modello non lo vede

Per questo il debug è NEL modulo

Il main di `search.py` stampa sempre i risultati **con gli score accanto** — senza score non si diagnostica nulla.

> La quarta riga è la coppia `rec-03` / `rec-04` di Modulo 2 · Lezione 7 che torna — stasera la incontrerete come **ricerca**.



Quanto tempo impiega una query

Il brute-force confronta la query con **ogni** vettore, uno per uno: più vettori ci sono, più tempo impiega. Con vettori come i nostri (1536 dimensioni):

n vettori	5.000	50.000	200.000
tempo per UNA query	8.9 ms	195.8 ms	**830.7 ms**

A 10 milioni di vettori

~41.5 secondi per UNA query — e ogni query rifà *tutto il lavoro da capo*. La crescita è lineare: nel brute-force non c'è ottimizzazione che la eviti.

Stasera va benissimo così

Coi nostri **48 testi** il tempo nemmeno si vede (sotto il millisecondo): il brute-force è **perfetto per capire**. Il problema esiste solo a scala — e ha una soluzione che vediamo **nella prossima lezione**.

> Da portare a casa: **più vettori = più tempo, in linea retta**. La parola chiave della prossima lezione è «**indice**».



🎤 Lettura guidata — si riapre il corpus

🎤 **LETTURA GUIDATA — si legge insieme**

`working/search.py · 10 minuti`

si torna alle slide
per l'esercizio

già scritta — la leggiamo riga per riga

`carica_corpus()` → `(docs, vectors, backend)`

npz + meta + testi, **nello stesso ordine** (docs[i] ↔ vettore i), e il backend letto da `corpus_meta.json` .

Prima mossa — il VOSTRO corpus della lezione scorsa

Chi ha i propri file di Modulo 2 · Lezione 7 li copia in `dataset/` (per gli altri la **riserva è già lì** — funziona uguale):

```
cp ../../lezione-07-
embeddings/scripts/working/corpus_* dataset/
```

Fatto quando

Lanciando `python working/search.py` la prima riga stampa **48 testi, 48 vettori e il backend**. Poi il file resta a voi: `search` e `search_mmr` li scrivete **voi**.



⌨ Esercizio 1 — le funzioni sono vostre

⌨ **TOCCA A VOI — docente in affiancamento**

`working/search.py · 40 minuti`

si torna alle slide
per il debrief

`TODO 1 · search(query, k)`

I **k testi più simili** alla query, dal più simile in giù, ognuno col suo **score**. La query si embedda col backend del corpus; la similarità è la cosine di Modulo 2 · Lezione 7.

`TODO 2 · search_mmr(query, k, lambda_)`

Come search, ma i k li sceglie **MMR** (la formula del capitolo 2). Con **λ=1.0 deve uscire il top-k identico**: è il vostro test. Se il tempo stringe: **prima search** — MMR si completa al debrief.

Prescritto poco, apposta

Fissati solo i **nomi**, le variabili del main (`QUERY_FACILE` , `QUERY_GENERICA` , `QUERY_DOPPIONI` , `K` , `LAMBDA`) e lo scopo: il **come è vostro**. Il main che prova le query e stampa gli score è **già scritto**: voi scrivete le due funzioni, poi **variate K e LAMBDA**.

Fatto quando

Su `QUERY_FACILE` la FAQ giusta esce **prima e staccata**; su `QUERY_DOPPIONI` MMR cambia faccia alla lista — e con **λ=1.0 le due liste coincidono** (provatelo: è il vostro test). Ogni lancio ≈ **$0.000002**. **Stretch:** una query vostra per famiglia (sensata, vaga, fuori dominio) e guardate le fasce.



📊 Al rientro — gli score delle prime due query

Le prime due query del main, in numeri: due storie diverse. Sempre top-3 con gli score sotto gli occhi.

Query	Top-3 (score)	Lettura
«Quanto costa la spedizione?»	`faq-02` **0.756** · poi 0.482, 0.475	**staccata**: query sana
«informazioni sulla scrivania»	0.595 · 0.573 · 0.573	**piatta**: query vaga, dice poco

Il top-k si è riempito COMUNQUE

In **entrambi** i casi sono usciti 3 risultati: è la **fascia degli score** a dirvi quale delle due ricerche vale qualcosa, non la lunghezza della lista.

0.756 **staccato** dal secondo = risposta trovata. Tre score **appiattiti** attorno a 0.58 = la query non sta dicendo niente di preciso.

> I vostri numeri possono differire di qualche millesimo (float32, versioni): contano le **fasce**, non le cifre.



📊 Al rientro — top-k vs MMR

`QUERY_DOPPIONI` — «problemi con il motore della scrivania», k=5, λ=0.5. Stessa query, due liste:

#	top-k		MMR (λ=0.5)	
1	`tic-02` rumore metallico	0.588	`tic-02` rumore metallico	0.588
2	`rec-04` motore rotto	0.574	`faq-03` **regolare l'altezza**	0.538
3	`faq-06` è rumorosa?	0.557	`tic-01` centralina spenta	0.543
4	`tic-01` centralina spenta	0.543	`faq-06` è rumorosa?	0.557
5	`tic-09` oscilla	0.540	`prod-02` **scheda LD-110**	0.443

Chi esce, chi entra

Escono i quasi-doppioni (`rec-04` , `tic-09` : ancora guasti e rumori), entrano una **FAQ** e un **prodotto**: la risposta copre più angoli.

MMR non è gratis

Il 5° MMR ha score **0.443** contro 0.540: paghi rilevanza per comprare diversità. Il cambio lo decide **λ** — a 1.0 le colonne coincidono (l'avete verificato).



⌨ Esercizio 2 — il query round

⌨ **QUERY ROUND — tre round, si discute insieme**

`il VOSTRO working/search.py · 25 minuti`

si chiude col reveal

`① LA QUERY`

Il **docente** ne dà UNA alla volta. Non le vedete in anticipo: *fa parte del gioco*.

`② ~5' DI PROVA`

Sostituite una query nel main e rilanciate: provatela con `search` e `search_mmr`, cercando i valori di **K e LAMBDA** che danno i risultati migliori.

`③ ~3' DI CONFRONTO`

Che **fascia** di score è uscita? Meglio top-k o MMR, qui? **Perché?** Poi round successivo.

Fatto quando

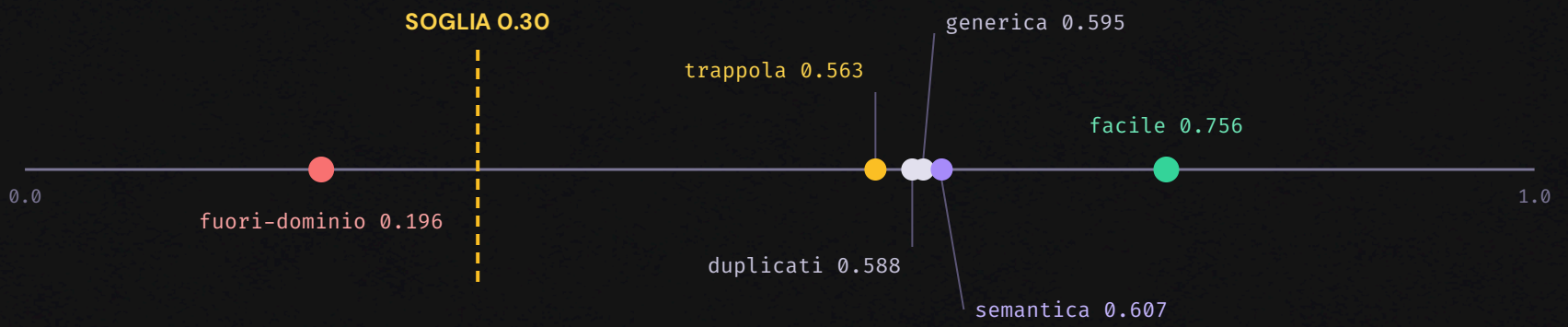
Per ogni round sapete dire: la **fascia** · **top-k o MMR** · i **k e λ** che avete scelto.

Niente file nuovi, niente TODO: è il **vostro software**, messo alla prova su query che non ha mai visto.



📊 Al rientro — le fasce e la soglia

Le **sei query della serata** su un'unica riga (score del top-1) — e la soglia disegnata nel **vuoto**:



La soglia NON salva dalle trappole

La trappola-negazione (0.563) resta **sopra** soglia: il threshold protegge dal fuori-dominio, non dalle trappole semantiche. **Tema ≠ verità non si filtra con un numero.**

Il **vuoto tra 0.196 e 0.563** si vede a occhio: la soglia va lì. E la fascia «semantica» fa **0.607 con parole quasi tutte diverse**: è il motivo per cui non usiamo il keyword matching.

Cosa portate a casa

🧰

`search.py` → `aikit/`

top-k + MMR, in prima persona. Da stasera nel **toolkit**, accanto a `embeddings.py`. Tutti, ora:

```
cp working/search.py
scaffolding/aikit/search.py
```

📏

Le fasce, e la vostra soglia

misurate nei round (0.756 → 0.196): la prova che il tuning si fa **sui numeri del proprio corpus**, non sui tutorial.

🧠

Le tre regole

stesso backend per query e corpus · gli score si leggono **a fasce** · rilevanza ≠ **diversità** (λ decide il compromesso).

Modulo 2 · Lezione 9: la stessa ricerca, indicizzata

I tempi di stasera: **830 ms su 200k vettori**, ~41.5 s su 10 milioni. Nella prossima lezione la stessa ricerca la fa un **vector DB** col suo indice (ANN) — e la vostra `search()` viene riscritta sopra.

Non cancellate il corpus in `dataset/` : si carica nel DB nella prossima lezione. (E al Modulo 2 · Lezione 10 arrivano i documenti lunghi: lì si spezza e si ri-cerca.)



Grazie

Prossima lezione — Modulo 2 · Lezione 9 · Vector Databases

